Rollin' Down the Biggest Hill: Analysis Draft

Model

In this model, a map is a directed graph $G=(V,E)$. Our test maps are n by n grids where each cell is a node with an elevation $el(v)$, and each cell has an edge going to each of its four neighbours. The weight of an edge is $w(u,v) = 0$ if $el(v) < el(u)$ (downhill: roll) and 1 otherwise (flat or uphill: push). The cost of a path is its number of pushes, and $\delta(s,t)$ is the true minimum cost of any path from s to t. Every edge has a reverse edge, so every node can reach every other node. All weights are 0 or 1, so none are negative, which is what Dijkstra's algorithm needs.

Algorithm

Our code is in *heely/dijkstra.py* and was modified from the code given in the class slides. Each node v has a current best guess $d[v]$ for its distance from the start s. It starts at *inf*, except $d[s] = 0$. A min-heap holds (distance, node) pairs and each step pops the node with the smallest distance and marks it settled. Then for each of its neighbours x, if going through v is cheaper than the current guess $d[x]$, we lower $d[x]$, remember v as x's parent, and push x onto the heap. We stop when the goal t is settled, and follow parents back from t to get the route.
Python's heap can't lower a value that is already inside it, so instead we push a new copy of the node and skip any old copy when it comes out later. This doesn't change the answer, it just leaves some extra entries in the heap.

Correctness

The main claim is that when a node is settled, its guess $d[v]$ is the true distance $\delta(s,v)$. This is CLRS Theorem 22.6 [1]. A rough version of the argument:

- $d[v]$ can never be too small, because every value it takes is the cost of a real path to v.
- Suppose some node u gets settled with a guess that is too big and u is the first time this happens. Take the real shortest path from s to u. Somewhere on it there is a first node y that isn't settled yet, right after a node x that is. Since x was settled correctly and we checked the edge from x to y, y's guess is already correct, so $d[y] = \delta(s,y)$.
- Weights aren't negative, so y is no further than u: $\delta(s,y) \le \delta(s,u) < d[u]$. That means y has a smaller guess than u, so the heap should have popped y first. But it popped u, which is a contradiction. So no node is ever settled with the wrong distance.

This also covers the old copies left in the heap since they get skipped, so can't cause a node to be settled twice. Since t is settled with $d[t] = \delta(s,t)$, and each parent step drops d by exactly that edge's weight, following parents from t back to s gives a route with exactly $\delta(s,t)$ pushes.

Running time

Dijkstra with a binary heap runs in $O((|V|+|E|) \log |V|)$ time [1]:

- Each node is settled once, and we only look at a node's edges when it is settled, so there are at most $|E|$ edge checks.
- Each edge check can push at most one entry, so the heap gets at most $|E| + 1$ pushes and pops.
- Each push or pop costs $O(\log |V|)$, which gives $O(|E| \log |V|)$, plus $O(|V|)$ to set up the arrays.

On our grids every node has at most 4 neighbours, so $|E| < 4|V|$, and with $|V| = n^2$ this is $O(n^2 \log n)$. The extra heap copies don't change this, because there are still at most $|E| + 1$ of them. Stopping when we reach t doesn't change the worst case, but in practice only the nodes closer to s than t get settled. Our preliminary results show about half of all nodes being settled on average. ALT, our second algorithm, is supposed to cut that down.

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. Chapter 22, Lemma 22.1 and Theorem 22.6.2